



SPARTAN
INTERNATIONAL
SR. SEC. SCHOOL

Affiliated to CBSE, New Delhi | Affiliation No.: 1930590 | Chennai - 600 123

FINAL COMPUTER SCIENCE PROJECT REPORT

Parking Lot Management System

Python CLI Application with SQLite Database

Modular Programming • SQLite Database • CLI Based System

Student Name	Joel Jebitto O
Class	XII
Section	A
Subject	Computer Science
Academic Session	2026–2027



Project Repository

<https://github.com/joeljebitto-dev/parking-lot-python-cli>

Acknowledgement and Abstract

I thank my Computer Science teacher, school, and parents for their guidance and support. This project is a Python-based parking-lot command line application that stores users, building layouts, parking spaces, occupied vehicles, and demo records using SQLite. The project demonstrates modular programming, SQL table design, input validation, relationships between entities, and command-line interaction.

Contents

Acknowledgement and Abstract	1
1 Project Overview	2
2 System Design Diagrams	3
3 Database Structure	5
4 Example Walkthrough of All Operations	7
5 Testing Plan and Conclusion	13
6 Complete Source Code	14

1 Project Overview

1.1 Problem Statement

Manual parking records can cause duplicate license plates, wrong vehicle-space allocation, slow vehicle lookup, and loss of information after the program closes. This project solves these problems through a persistent Python CLI backed by SQLite.

1.2 Objectives

1. To use SQLite for persistent storage of users and parking-space data.
2. To validate email, phone number, name, status, and license plate input.
3. To allocate the first compatible free space according to vehicle type.
4. To provide dashboard, search, removal, clear-building, and demo-seeding features.

1.3 Scope and Requirements

Hardware/Software	Python 3.x, SQLite, Rich library, Windows/Linux/macOS, 4 GB RAM recommended.
Main modules	Models, validation, database schema creation, building layouts, user repository, parking-space repository, CLI, demo seed.
Main features	Building selection, user CRUD, parking allocation, occupied-space listing, license-plate search, vehicle removal, clear building, seed demo data, clean exit.
Persistence	Runtime data is stored in <code>parking_lot/parking_lot.db</code> ; parked vehicles remain after restart.

1.4 Menu Plan

Menu	Feature	Purpose
1	Dashboard	View available, occupied, and total spaces.
2	List users	Display all saved users.
3	Create user	Insert a new user after validation.
4	Edit user	Update selected fields of an existing user.
5	Delete user	Remove user and clear related occupied spaces.
6	Park vehicle	Allocate first compatible free slot.
7	Occupied spaces	List all currently occupied spaces.
8	Find vehicle by plate	Search vehicle by normalized license plate.
9	Remove vehicle	Remove by space id or license plate.
10	Show structures	Display available building layouts.
11	Switch building	Change active building.
12	Clear selected building	Reset occupied spaces in the current building.
13	Seed demo data	Create demo users and demo parked vehicles.
14	Exit	Stop the program cleanly.

2 System Design Diagrams

2.1 Architecture Diagram

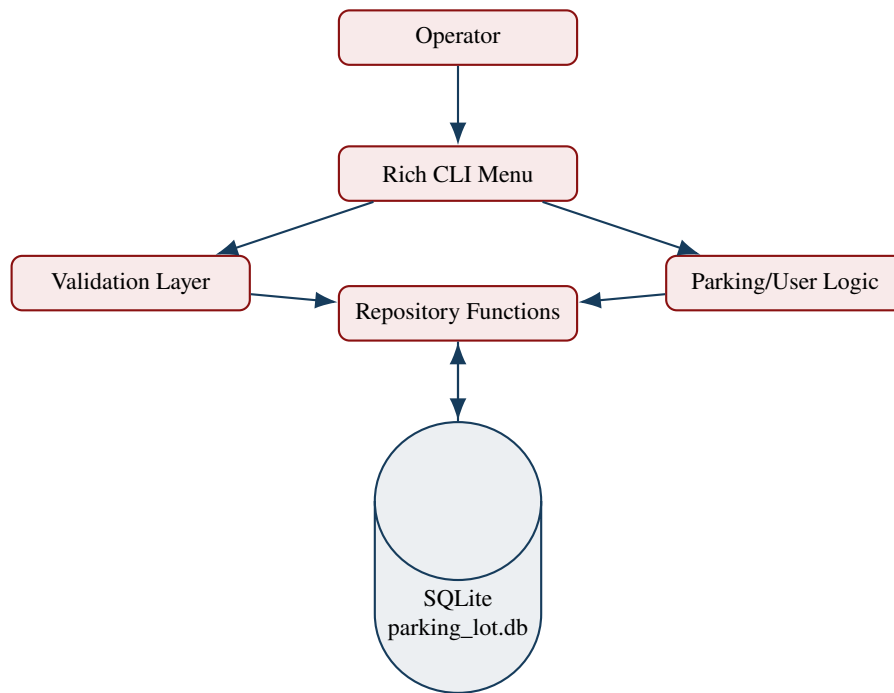


Figure 1: Layered system architecture

2.2 Overall CLI Operation Flowchart

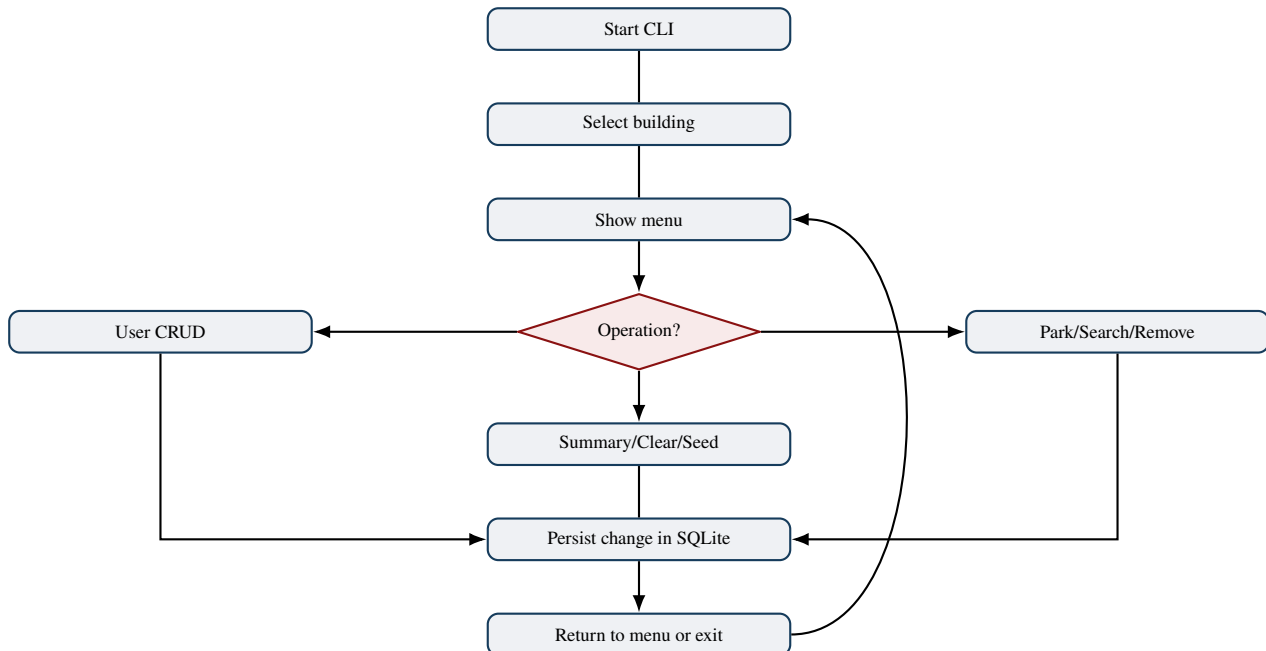


Figure 2: Overall CLI operation flow

2.3 Park Vehicle Flowchart

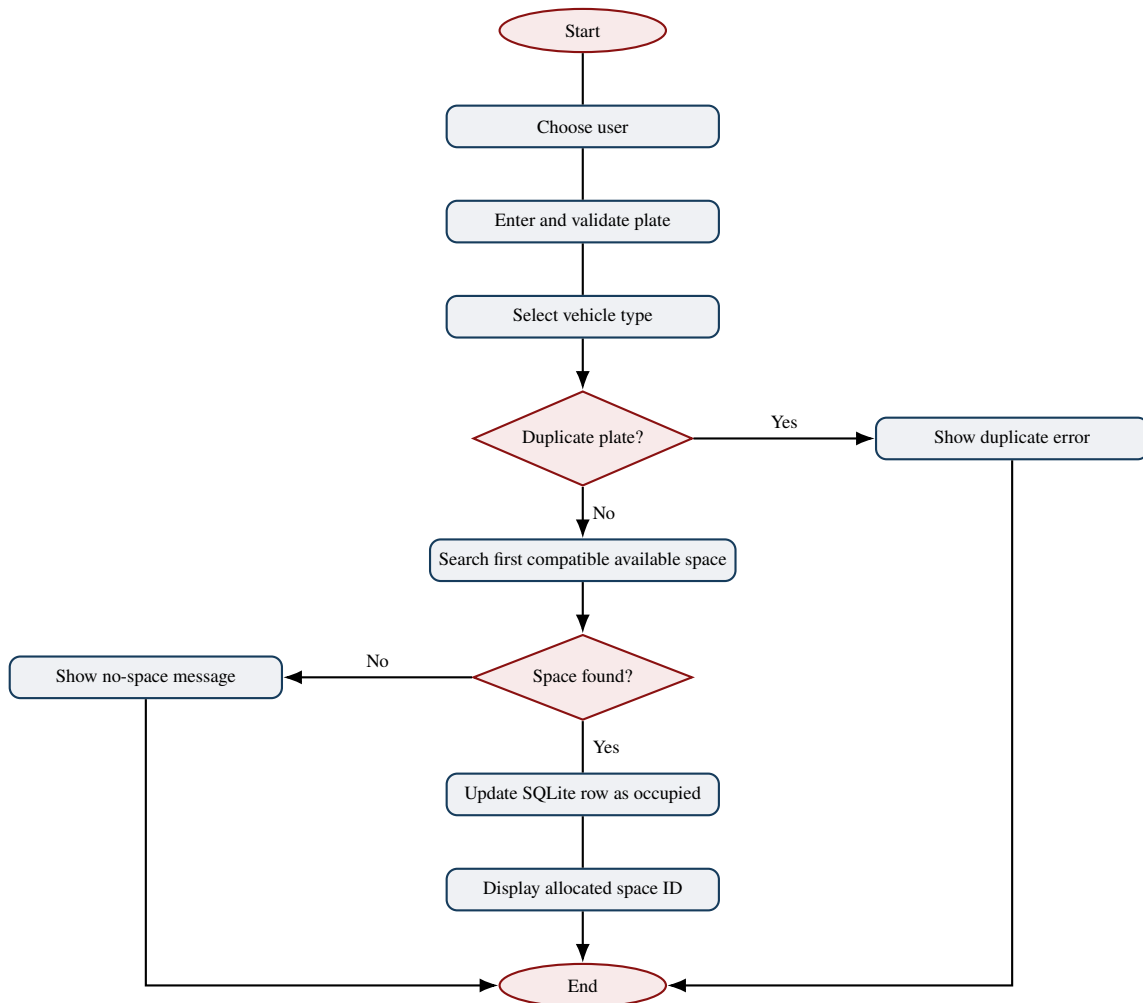


Figure 3: Vehicle parking flowchart

2.4 Remove Vehicle Flowchart

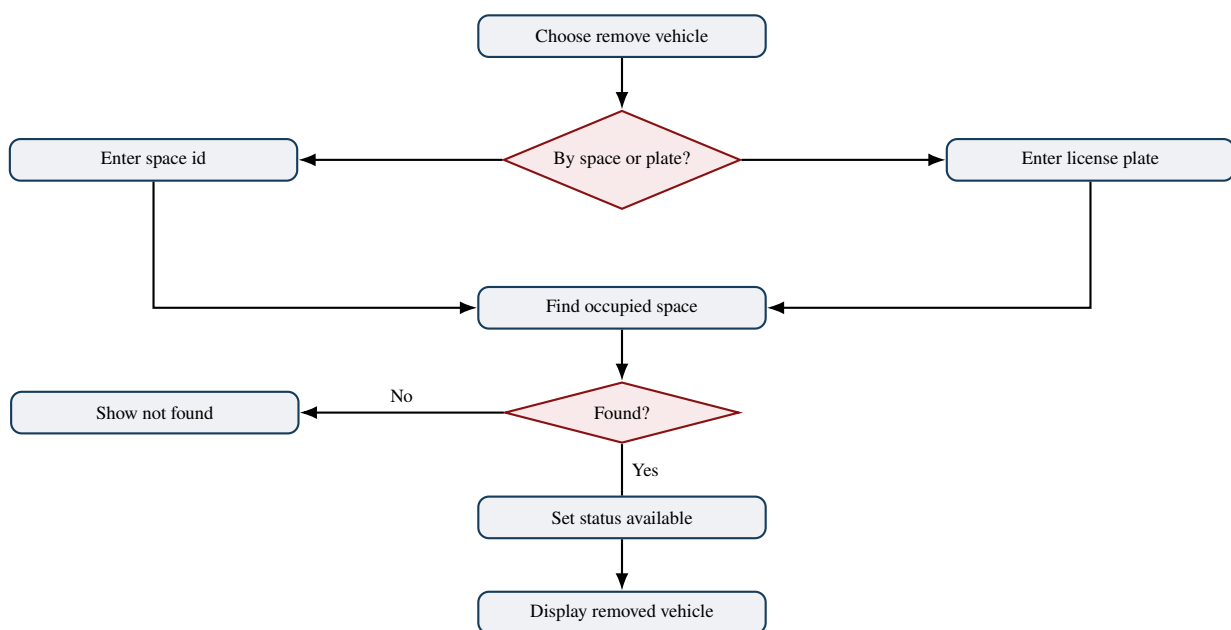


Figure 4: Vehicle removal flowchart

3 Database Structure

3.1 Entity Relationship Diagram

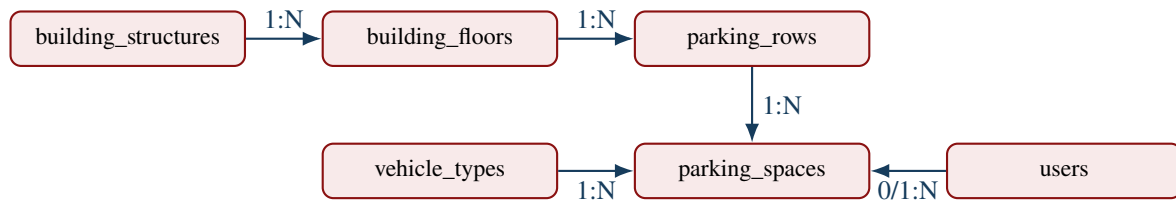


Figure 5: SQLite database relationships

3.2 Detailed Table Structure

Table	Primary Key	Foreign Keys	Purpose and Constraints
users	id	None	Stores first name, last name, email, phone, status, and created date. Email is unique and case-insensitive. Status is restricted to active, inactive, or pending.
vehicle_types	id	None	Master table for BIKE, CAR_MINI, CAR_SUV, TRUCK_MINI, and TRUCK_LARGE. Code is unique.
building_structures	id	None	Stores named layouts such as small, mixed, truck_friendly, and very_large_complex. Name is unique.
building_floors	id	building_structure_id	Stores floor numbers for each building. Building and floor number are unique together.
parking_rows	id	building_floor_id	Stores row numbers for each floor. Floor and row number are unique together.
parking_spaces	id	parking_row_id, vehicle_type_id, user_id	Stores individual parking slots. Status is available or occupied. Available spaces use NULL user and plate. Occupied license plates are unique.

3.3 Column-Level Data Dictionary

Table	Column	Type	Meaning
users	id	INTEGER	Primary key used to identify a user.
users	first_name, last_name	TEXT	User identity fields.
users	email	TEXT	Unique case-insensitive email address.
users	phone	TEXT	Contact number.
users	status	TEXT	Must be active, inactive, or pending.
vehicle_types	id, code, label	INT/TEXT	Master values used for compatible parking-space matching.
building_structures	id, name	INT/TEXT	Named building layout.
building_floors	building_structure_id	INTEGER	Links a floor to a building.
parking_rows	building_floor_id	INTEGER	Links a row to a floor.
parking_spaces	parking_row_id	INTEGER	Links a space to a row.
parking_spaces	vehicle_type_id	INTEGER	Defines what vehicle type can park there.
parking_spaces	status	TEXT	available or occupied.
parking_spaces	user_id, license_plate	INT/TEXT	Occupant details when a vehicle is parked.

3.4 SQLite Schema

```

1 CREATE TABLE users (
2   id INTEGER PRIMARY KEY,
3   first_name TEXT NOT NULL,
4   last_name TEXT NOT NULL,

```

```
5      email TEXT NOT NULL COLLATE NOCASE UNIQUE,
6      phone TEXT NOT NULL,
7      status TEXT NOT NULL CHECK (status IN ('active', 'inactive', 'pending')),
8      created_at TEXT NOT NULL
9  );
10
11 CREATE TABLE vehicle_types (
12     id INTEGER PRIMARY KEY,
13     code TEXT NOT NULL UNIQUE,
14     label TEXT NOT NULL
15 );
16
17 CREATE TABLE building_structures (
18     id INTEGER PRIMARY KEY,
19     name TEXT NOT NULL UNIQUE
20 );
21
22 CREATE TABLE building_floors (
23     id INTEGER PRIMARY KEY,
24     building_structure_id INTEGER NOT NULL,
25     floor_number INTEGER NOT NULL,
26     UNIQUE (building_structure_id, floor_number),
27     FOREIGN KEY (building_structure_id) REFERENCES building_structures (id) ON DELETE CASCADE
28 );
29
30 CREATE TABLE parking_rows (
31     id INTEGER PRIMARY KEY,
32     building_floor_id INTEGER NOT NULL,
33     row_number INTEGER NOT NULL,
34     UNIQUE (building_floor_id, row_number),
35     FOREIGN KEY (building_floor_id) REFERENCES building_floors (id) ON DELETE CASCADE
36 );
37
38 CREATE TABLE parking_spaces (
39     id INTEGER PRIMARY KEY,
40     parking_row_id INTEGER NOT NULL,
41     space_id TEXT NOT NULL,
42     spot_number INTEGER NOT NULL,
43     vehicle_type_id INTEGER NOT NULL,
44     status TEXT NOT NULL DEFAULT 'available' CHECK (status IN ('available', 'occupied')),
45     user_id INTEGER,
46     license_plate TEXT COLLATE NOCASE,
47     UNIQUE (parking_row_id, spot_number),
48     FOREIGN KEY (parking_row_id) REFERENCES parking_rows (id) ON DELETE CASCADE,
49     FOREIGN KEY (vehicle_type_id) REFERENCES vehicle_types (id),
50     FOREIGN KEY (user_id) REFERENCES users (id) ON DELETE SET NULL
51 );
52
53 CREATE UNIQUE INDEX idx_occupied_license_plate
54 ON parking_spaces (license_plate)
55 WHERE status = 'occupied' AND license_plate IS NOT NULL;
```

Listing 1: Full SQLite schema

4 Example Walkthrough of All Operations

This section presents actual raw terminal-output screenshots captured by running the Python CLI. Each page contains only two output screens so that the terminal output remains readable in print.

Startup and dashboard

```

Actual terminal output - Parking Lot CLI

$ python3 main.py # startup, building selection, main menu

Parking Lot Manager
  Choose a building, manage users, and persist parked vehicles.

Select the building to operate on first.
      Parking Building Structures

```

Option	Name	Spaces	Breakdown
1	small	5	2 Bike, 2 Mini Car, 1 SUV Car
2	mixed	10	2 Bike, 3 Mini Car, 2 SUV Car, 2 Mini Truck, 1 Large Truck
3	truck_friendly	8	1 Mini Car, 1 SUV Car, 3 Mini Truck, 3 Large Truck
4	very_large_complex	295	100 Bike, 100 Mini Car, 50 SUV Car, 30 Mini Truck, 15 Large Truck

```

Structure name or number:
} Building: small    Available: 5    Occupied: 0    Total: 5

1    Dashboard
2    List users
...

```

Figure 6: Start, building selection, and main menu

```

Actual terminal output - Parking Lot CLI

$ python3 main.py # dashboard

Parking Summary

```

Vehicle Type	Available	Occupied	Total
Bike	2	0	2
Mini Car	2	0	2
SUV Car	1	0	1
Mini Truck	0	0	0
Large Truck	0	0	0

```

No occupied spaces.
      Parking Lot CLI
} Building: small    Available: 5    Occupied: 0    Total: 5

1    Dashboard
2    List users
3    Create user
4    Edit user
5    Delete user
6    Park vehicle
7    Occupied spaces
8    Find vehicle by plate
...

```

Figure 7: Dashboard and parking summary

User management operations

```

Actual terminal output - Parking Lot CLI

$ python3 main.py # create user and list users

Create User
First name: Last name: Email: Phone: Status [active/inactive/pending] (active): Created user 1: Joel Student

Parking Lot CLI
Building: mixed Available: 10 Occupied: 0 Total: 10

1 Dashboard
2 List users
3 Create user
4 Edit user
5 Delete user
6 Park vehicle
7 Occupied spaces
8 Find vehicle by plate
9 Remove vehicle
10 Show building structures
11 Switch building
12 Clear selected building
13 Seed demo data
14 Exit
...

```

Figure 8: Create user and list users

```

Actual terminal output - Parking Lot CLI

$ python3 main.py # edit user, delete user, list users

Press Enter to keep the current value.
First name [Aarav]: Last name [Kumar]: Email : Phone [+91-90000-00001]: Status [active/inactive/pending] (active): Updated user 1: Aarav Kumar

Parking Lot CLI
Building: mixed Available: 10 Occupied: 0 Total: 10

1 Dashboard
2 List users
3 Create user
4 Edit user
5 Delete user
6 Park vehicle
7 Occupied spaces
8 Find vehicle by plate
9 Remove vehicle
10 Show building structures
11 Switch building
12 Clear selected building
13 Seed demo data
14 Exit

Choose an option [1/2/3/4/5/6/7/8/9/10/11/12/13/14]:
...

```

Figure 9: Edit user and delete user

Parking and search operations

```

Actual terminal output - Parking Lot CLI

$ python3 main.py # park vehicle

Vehicle Types


| Option | Vehicle Type |
|--------|--------------|
| 1      | Bike         |
| 2      | Mini Car     |
| 3      | SUV Car      |
| 4      | Mini Truck   |
| 5      | Large Truck  |



Vehicle type number (blank to cancel): _____ Vehicle Parked _____

Allocated F1-R1-S1
Vehicle: Bike
Plate: KA01AB1234
User: Priya Nair

Parking Lot CLI
Building: mixed Available: 9 Occupied: 1 Total: 10

1 Dashboard
2 List users
...

```

Figure 10: Park vehicle operation

```

Actual terminal output - Parking Lot CLI

$ python3 main.py # occupied spaces and find vehicle

Occupied Spaces


| Space    | Vehicle | Plate      | User        | User ID |
|----------|---------|------------|-------------|---------|
| F1-R1-S1 | Bike    | TN10AB1234 | Rohan Mehta | 1       |



Parking Lot CLI
Building: mixed Available: 9 Occupied: 1 Total: 10

1 Dashboard
2 List users
3 Create user
4 Edit user
5 Delete user
6 Park vehicle
7 Occupied spaces
8 Find vehicle by plate
9 Remove vehicle
10 Show building structures
11 Switch building
12 Clear selected building
13 Seed demo data
...

```

Figure 11: Occupied spaces and license-plate search

Removal and building-structure operations

```

Actual terminal output - Parking Lot CLI

$ python3 main.py # remove vehicle by space id

Remove by [space/plate/cancel] (space):          Occupied Spaces


| Space    | Vehicle | Plate      | User       | User ID |
|----------|---------|------------|------------|---------|
| F1-R1-S1 | Bike    | DL09CD7777 | Ananya Rao | 1       |


Space id (blank to cancel): Removed Bike DL09CD7777 from F1-R1-S1 for Ananya Rao.
Parking Lot CLI
Building: mixed    Available: 10    Occupied: 0    Total: 10

1  Dashboard
2  List users
3  Create user
4  Edit user
5  Delete user
6  Park vehicle
7  Occupied spaces
8  Find vehicle by plate
9  Remove vehicle
10 Show building structures
11 Switch building
12 Clear selected building
...

```

Figure 12: Remove vehicle by space or license plate

```

Actual terminal output - Parking Lot CLI

$ python3 main.py # show structures and switch building

Parking Building Structures


| Option | Name               | Spaces | Breakdown                                                         |
|--------|--------------------|--------|-------------------------------------------------------------------|
| 1      | small              | 5      | 2 Bike, 2 Mini Car, 1 SUV Car                                     |
| 2      | mixed              | 10     | 2 Bike, 3 Mini Car, 2 SUV Car, 2 Mini Truck, 1 Large Truck        |
| 3      | truck_friendly     | 8      | 1 Mini Car, 1 SUV Car, 3 Mini Truck, 3 Large Truck                |
| 4      | very_large_complex | 295    | 100 Bike, 100 Mini Car, 50 SUV Car, 30 Mini Truck, 15 Large Truck |


Structure name or number: Parking Lot CLI
Building: small    Available: 5    Occupied: 0    Total: 5

1  Dashboard
2  List users
3  Create user
4  Edit user
5  Delete user
6  Park vehicle
...

```

Figure 13: Show structures and switch building

Administrative and validation outputs

```

Actual terminal output - Parking Lot CLI

$ python3 main.py # seed demo data and clear building

Reset demo users and demo parking occupancy? [y/n] (n): Seeded 10 demo users and 30 occupied spaces.
Parking Lot CLI
Building: mixed Available: 0 Occupied: 10 Total: 10

1 Dashboard
2 List users
3 Create user
4 Edit user
5 Delete user
6 Park vehicle
7 Occupied spaces
8 Find vehicle by plate
9 Remove vehicle
10 Show building structures
11 Switch building
12 Clear selected building
13 Seed demo data
14 Exit

Choose an option [1/2/3/4/5/6/7/8/9/10/11/12/13/14]: Clear all occupied spaces in mixed? [y/n] (n): Clear
ed 10 occupied spaces.
Parking Lot CLI
...

```

Figure 14: Clear selected building and seed demo data

```

Actual terminal output - Parking Lot CLI

$ python3 main.py # validation errors and exit

Create User
First name: Last name: Email: Phone: Status [active/inactive/pending] (active): Email must be a valid ema
il address.
Try again? [y/n] (y):
Parking Lot CLI
Building: mixed Available: 10 Occupied: 0 Total: 10

1 Dashboard
2 List users
3 Create user
4 Edit user
5 Delete user
6 Park vehicle
7 Occupied spaces
8 Find vehicle by plate
9 Remove vehicle
10 Show building structures
11 Switch building
12 Clear selected building
13 Seed demo data
14 Exit

...

```

Figure 15: Validation errors and exit

4.1 Operation Walkthrough Matrix

Menu	Operation	Input Example	Expected Output
1	Dashboard	Choose 1	Summary by vehicle type and occupied-space preview.
2	List users	Choose 2	Table of existing users or no-users message.
3	Create user	Name, email, phone, status	User record saved in SQLite.
4	Edit user	User id and changed fields	Existing user updated.
5	Delete user	User id and confirmation	User removed and related occupied slots cleared.
6	Park vehicle	User id, plate, vehicle type	First compatible free slot allocated.
7	Occupied spaces	Choose 7	Table with space, vehicle, plate, user, and user id.
8	Find vehicle	License plate	Building, space, vehicle type, plate, and user details.
9	Remove vehicle	Space id or plate	Occupancy cleared and removed vehicle shown.
10	Show structures	Choose 10	Layout list with available building structures.
11	Switch building	Structure number/name	Active building changed.
12	Clear building	Confirmation	All occupied spaces in selected building cleared.
13	Seed demo data	Confirmation	10 demo users and 30 occupied spaces generated.
14	Exit	Choose 14	Program terminates cleanly.

5 Testing Plan and Conclusion

Test Area	Expected Result
User validation	Invalid email, blank name, and invalid status are rejected.
Duplicate user	Existing email is rejected by validation/database constraint.
Parking allocation	Correct first available compatible slot is allocated.
Duplicate vehicle	Same occupied license plate is rejected.
Search and removal	Vehicle can be found and removed by license plate or space id.
Persistence	Data remains after restarting because it is stored in SQLite.

5.1 Limitations and Future Scope

Limitations

- CLI-based, no graphical interface.
- No fee calculation or payment module.
- No ticket generation in the current version.
- Simple first-compatible-space strategy.

Future Scope

- Add GUI or web dashboard.
- Add entry time, exit time, and fee calculation.
- Add QR-code parking tickets.
- Add admin login and report generation.

5.2 Conclusion

The Parking Lot Management System satisfies the main goal of a practical, modular Python project. It demonstrates classes, functions, validation, SQL schema design, persistent storage, command-line menus, and real-world problem solving. The project is suitable for school-level practical demonstration and viva explanation.

6 Complete Source Code

The following source listing contains the application code files included in the project folder.

6.1 Project File Tree

```
main.py
requirements.txt
parking_lot/
  __init__.py
  models.py
  validation.py
  structures.py
  database.py
  user_repository.py
  parking_space_repository.py
  building.py
  demo_seed.py
  cli.py
```

6.2 main.py

```
1 from parking_lot.cli import run_cli
2
3
4 if __name__ == "__main__":
5     run_cli()
```

6.3 requirements.txt

```
1 rich
```

6.4 parking_lot/__init__.py

```
1 """Parking Lot Management System package."""
```

6.5 parking_lot/models.py

```
1 from __future__ import annotations
2
3 from dataclasses import dataclass
4 from enum import Enum
5 from typing import Optional
6
7
8 @dataclass
9 class User:
10     user_id: int
11     first_name: str
12     last_name: str
13     email: str
14     phone: str
15     status: str
16     created_at: str
17
18     def __post_init__(self):
19         self.user_id = int(self.user_id)
20
21     def get_full_name(self):
22         return f"{self.first_name} {self.last_name}"
23
24     def get_user_id(self):
25         return self.user_id
26
27 class VehicleType(Enum):
28     BIKE = 1
29     CAR_MINI = 2
30     CAR_SUV = 3
31     TRUCK_MINI = 4
32     TRUCK_LARGE = 5
33
34     @classmethod
35     def from_value(cls, value):
36
```

```

37     if isinstance(value, VehicleType):
38         return value
39     normalized = str(value).strip().upper()
40     if normalized in cls.__members__:
41         return cls[normalized]
42     raise ValueError(f"Unsupported vehicle type: {value}")
43
44
45 @dataclass
46 class Vehicle:
47     vehicle_type: VehicleType
48     user: Optional[User] = None
49     license_plate: str = ""
50
51     def __post_init__(self):
52         self.vehicle_type = VehicleType.from_value(self.vehicle_type)
53         self.license_plate = self.license_plate.strip().upper()
54
55
56 @dataclass
57 class ParkingSpace:
58     building_name: str
59     space_id: str
60     floor: int
61     row: int
62     spot: int
63     vehicle_type: VehicleType
64     status: str = "available"
65     parked_vehicle: Optional[Vehicle] = None
66
67     def __post_init__(self):
68         self.floor = int(self.floor)
69         self.row = int(self.row)
70         self.spot = int(self.spot)
71         self.vehicle_type = VehicleType.from_value(self.vehicle_type)
72         self.status = self.status or "available"
73
74     def is_available(self):
75         return self.status == "available" and self.parked_vehicle is None
76
77     def can_park(self, vehicle):
78         return self.is_available() and vehicle.vehicle_type == self.vehicle_type
79
80     def park_vehicle(self, vehicle):
81         if not self.can_park(vehicle):
82             return False
83         self.parked_vehicle = vehicle
84         self.status = "occupied"
85         return True
86
87     def remove_vehicle(self):
88         vehicle = self.parked_vehicle
89         self.parked_vehicle = None
90         self.status = "available"
91         return vehicle
92
93     def get_user_id(self):
94         if self.parked_vehicle is None or self.parked_vehicle.user is None:
95             return ""
96         return self.parked_vehicle.user.get_user_id()
97
98     def get_license_plate(self):
99         if self.parked_vehicle is None:
100             return ""
101         return self.parked_vehicle.license_plate

```

6.6 parking_lot/validation.py

```

1 import re
2
3 VALID_USER_STATUSES = {"active", "inactive", "pending"}
4
5
6 def normalize_email(email):
7     return email.strip().lower()
8
9
10 def normalize_status(status):
11     return status.strip().lower()
12

```

```

13
14 def normalize_license_plate(license_plate):
15     return " ".join(license_plate.strip().upper().split())
16
17
18 def validate_email(email):
19     normalized_email = normalize_email(email)
20     if not re.fullmatch(r"^[^\s]+@[^\s]+\.[^\s]+$", normalized_email):
21         raise ValueError("Email must be a valid email address.")
22     return normalized_email
23
24
25 def validate_phone(phone):
26     normalized_phone = phone.strip()
27     if not normalized_phone:
28         raise ValueError("Phone cannot be blank.")
29     return normalized_phone
30
31
32 def validate_status(status):
33     normalized_status = normalize_status(status)
34     if normalized_status not in VALID_USER_STATUSES:
35         raise ValueError("Status must be active, inactive, or pending.")
36     return normalized_status
37
38
39 def validate_name(value, field_name):
40     normalized_value = value.strip()
41     if not normalized_value:
42         raise ValueError(f"{field_name} cannot be blank.")
43     return normalized_value
44
45
46 def validate_license_plate(license_plate):
47     normalized_license_plate = normalize_license_plate(license_plate)
48     if not re.fullmatch(r"[A-Z0-9 -]{2,20}", normalized_license_plate):
49         raise ValueError(
50             "License plate must be 2-20 characters using letters, numbers, spaces, or hyphens."
51         )
52     return normalized_license_plate

```

6.7 parking_lot/structures.py

```

1 from collections import Counter
2 from copy import deepcopy
3 from parking_lot.models import VehicleType
4
5
6 def build_parking_building_structure(vehicle_types, spaces_per_row=10, rows_per_floor=10):
7     structure = []
8     spaces_per_floor = spaces_per_row * rows_per_floor
9     for floor_start in range(0, len(vehicle_types), spaces_per_floor):
10         floor_vehicle_types = vehicle_types[floor_start: floor_start + spaces_per_floor]
11         floor = []
12         for row_start in range(0, len(floor_vehicle_types), spaces_per_row):
13             floor.append(floor_vehicle_types[row_start: row_start + spaces_per_row])
14         structure.append(floor)
15     return structure
16
17
18 SMALL_PARKING_BUILDING = [
19     [[VehicleType.BIKE, VehicleType.BIKE, VehicleType.CAR_MINI],
20      [VehicleType.CAR_MINI, VehicleType.CAR_SUV]],
21 ]
22
23 MIXED_PARKING_BUILDING = [
24     [[VehicleType.BIKE, VehicleType.BIKE, VehicleType.CAR_MINI],
25      [VehicleType.CAR_MINI, VehicleType.CAR_SUV, VehicleType.CAR_SUV]],
26     [[VehicleType.TRUCK_MINI, VehicleType.CAR_MINI],
27      [VehicleType.TRUCK_LARGE, VehicleType.TRUCK_MINI]],
28 ]
29
30 TRUCK_FRIENDLY_PARKING_BUILDING = [
31     [[VehicleType.TRUCK_MINI, VehicleType.TRUCK_MINI],
32      [VehicleType.TRUCK_LARGE, VehicleType.TRUCK_LARGE]],
33     [[VehicleType.CAR_SUV, VehicleType.TRUCK_MINI],
34      [VehicleType.TRUCK_LARGE, VehicleType.CAR_MINI]],
35 ]
36
37 VERY_LARGE_COMPLEX_PARKING_BUILDING = build_parking_building_structure(

```

```

38     [VehicleType.BIKE] * 100
39     + [VehicleType.CAR_MINI] * 100
40     + [VehicleType.CAR_SUV] * 50
41     + [VehicleType.TRUCK_MINI] * 30
42     + [VehicleType.TRUCK_LARGE] * 15
43 )
44
45 PARKING_BUILDING_STRUCTURES = {
46     "small": SMALL_PARKING_BUILDING,
47     "mixed": MIXED_PARKING_BUILDING,
48     "truck_friendly": TRUCK_FRIENDLY_PARKING_BUILDING,
49     "very_large_complex": VERY_LARGE_COMPLEX_PARKING_BUILDING,
50 }
51
52
53 def get_structure_names():
54     return list(PARKING_BUILDING_STRUCTURES.keys())
55
56
57 def get_parking_building_structure(name):
58     if name not in PARKING_BUILDING_STRUCTURES:
59         raise ValueError(f"Unknown parking building structure: {name}")
60     return deepcopy(PARKING_BUILDING_STRUCTURES[name])
61
62
63 def count_vehicle_types(structure):
64     return Counter(vehicle_type for floor in structure for row in floor for vehicle_type in row)

```

6.8 parking_lot/database.py

```

1 from __future__ import annotations
2
3 import sqlite3
4 from pathlib import Path
5 from parking_lot.models import VehicleType
6 from parking_lot.structures import PARKING_BUILDING_STRUCTURES
7
8 DEFAULT_DB_PATH = Path(__file__).resolve().parent / "parking_lot.db"
9
10
11 def resolve_db_path(db_path=None):
12     return Path(db_path) if db_path is not None else DEFAULT_DB_PATH
13
14
15 def get_connection(db_path=None):
16     resolved_path = resolve_db_path(db_path)
17     resolved_path.parent.mkdir(parents=True, exist_ok=True)
18     connection = sqlite3.connect(resolved_path)
19     connection.row_factory = sqlite3.Row
20     connection.execute("PRAGMA foreign_keys = ON")
21     initialize_database(connection)
22     return connection
23
24
25 def initialize_database(connection):
26     connection.executescript("""
27         CREATE TABLE IF NOT EXISTS users (
28             id INTEGER PRIMARY KEY,
29             first_name TEXT NOT NULL,
30             last_name TEXT NOT NULL,
31             email TEXT NOT NULL COLLATE NOCASE UNIQUE,
32             phone TEXT NOT NULL,
33             status TEXT NOT NULL CHECK (status IN ('active', 'inactive', 'pending')),
34             created_at TEXT NOT NULL
35         );
36         CREATE TABLE IF NOT EXISTS vehicle_types (
37             id INTEGER PRIMARY KEY,
38             code TEXT NOT NULL UNIQUE,
39             label TEXT NOT NULL
40         );
41         CREATE TABLE IF NOT EXISTS building_structures (
42             id INTEGER PRIMARY KEY,
43             name TEXT NOT NULL UNIQUE
44         );
45         CREATE TABLE IF NOT EXISTS building_floors (
46             id INTEGER PRIMARY KEY,
47             building_structure_id INTEGER NOT NULL,
48             floor_number INTEGER NOT NULL,
49             UNIQUE (building_structure_id, floor_number),
50             FOREIGN KEY (building_structure_id) REFERENCES building_structures (id) ON DELETE CASCADE

```

```

51     );
52     CREATE TABLE IF NOT EXISTS parking_rows (
53         id INTEGER PRIMARY KEY,
54         building_floor_id INTEGER NOT NULL,
55         row_number INTEGER NOT NULL,
56         UNIQUE (building_floor_id, row_number),
57         FOREIGN KEY (building_floor_id) REFERENCES building_floors (id) ON DELETE CASCADE
58     );
59     CREATE TABLE IF NOT EXISTS parking_spaces (
60         id INTEGER PRIMARY KEY,
61         parking_row_id INTEGER NOT NULL,
62         space_id TEXT NOT NULL,
63         spot_number INTEGER NOT NULL,
64         vehicle_type_id INTEGER NOT NULL,
65         status TEXT NOT NULL DEFAULT 'available' CHECK (status IN ('available', 'occupied')),
66         user_id INTEGER,
67         license_plate TEXT COLLATE NOCASE,
68         UNIQUE (parking_row_id, spot_number),
69         FOREIGN KEY (parking_row_id) REFERENCES parking_rows (id) ON DELETE CASCADE,
70         FOREIGN KEY (vehicle_type_id) REFERENCES vehicle_types (id),
71         FOREIGN KEY (user_id) REFERENCES users (id) ON DELETE SET NULL
72     );
73     CREATE UNIQUE INDEX IF NOT EXISTS idx_occupied_license_plate
74         ON parking_spaces (license_plate)
75         WHERE status = 'occupied' AND license_plate IS NOT NULL;
76     """
77     seed_vehicle_types(connection)
78     seed_building_structures(connection)
79     connection.commit()
80
81
82 def seed_vehicle_types(connection):
83     for vehicle_type in VehicleType:
84         connection.execute(
85             "INSERT OR IGNORE INTO vehicle_types (id, code, label) VALUES (?, ?, ?)",
86             (vehicle_type.value, vehicle_type.name, vehicle_type.name.replace("-", " ").title()),
87         )
88
89
90 def seed_building_structures(connection):
91     for building_name, structure in PARKING_BUILDING_STRUCTURES.items():
92         building_id = get_or_create_id(connection, "building_structures", "name", building_name)
93         for floor_index, floor in enumerate(structure, start=1):
94             floor_id = get_or_create_scoped_id(connection, "building_floors", "building_structure_id", building_id,
95             "floor_number", floor_index)
96             for row_index, row in enumerate(floor, start=1):
97                 row_id = get_or_create_scoped_id(connection, "parking_rows", "building_floor_id", floor_id,
98                 "row_number", row_index)
99                 for spot_index, vehicle_type in enumerate(row, start=1):
100                     connection.execute(
101                         """
102                         INSERT OR IGNORE INTO parking_spaces
103                         (parking_row_id, space_id, spot_number, vehicle_type_id, status)
104                         VALUES (?, ?, ?, ?, 'available')
105                         """
106                     )
107                     (row_id, create_space_id(floor_index, row_index, spot_index), spot_index, vehicle_type.value),
108
109
110 def get_or_create_id(connection, table_name, column_name, value):
111     connection.execute(f"INSERT OR IGNORE INTO {table_name} ({column_name}) VALUES (?)", (value,))
112     return connection.execute(f"SELECT id FROM {table_name} WHERE {column_name} = ?", (value,)).fetchone()["id"]
113
114
115 def get_or_create_scoped_id(connection, table_name, scope_column, scope_value, value_column, value):
116     connection.execute(
117         f"INSERT OR IGNORE INTO {table_name} ({scope_column}, {value_column}) VALUES (?, ?)",
118         (scope_value, value),
119     )
120     return connection.execute(
121         f"SELECT id FROM {table_name} WHERE {scope_column} = ? AND {value_column} = ?",
122         (scope_value, value),
123     ).fetchone()["id"]
124
125 def create_space_id(floor, row, spot):
126     return f"F{floor}-R{row}-S{spot}"

```

6.9 parking_lot/user_repository.py

```

1 import sqlite3
2 from contextlib import closing
3 from datetime import date
4 from parking_lot.database import get_connection, resolve_db_path
5 from parking_lot.models import User
6 from parking_lot.validation import validate_email, validate_name, validate_phone, validate_status
7
8
9 def load_users(db_path=None):
10     resolved_db_path = resolve_db_path(db_path)
11     with closing(get_connection(resolved_db_path)) as connection:
12         rows = connection.execute(
13             "SELECT id, first_name, last_name, email, phone, status, created_at FROM users ORDER BY id"
14         ).fetchall()
15     return [row_to_user(row) for row in rows]
16
17
18 def create_user(first_name, last_name, email, phone, status="active", created_at=None, user_id=None, db_path=None):
19     resolved_db_path = resolve_db_path(db_path)
20     users = load_users(db_path=resolved_db_path)
21     user = build_validated_user(
22         user_id or get_next_user_id(db_path=resolved_db_path),
23         first_name, last_name, email, phone, status, created_at or date.today().isoformat(), users,
24     )
25     with closing(get_connection(resolved_db_path)) as connection:
26         try:
27             connection.execute(
28                 """
29                 INSERT INTO users (id, first_name, last_name, email, phone, status, created_at)
30                 VALUES (?, ?, ?, ?, ?, ?, ?)
31                 """,
32                 user_to_record(user),
33             )
34             connection.commit()
35         except sqlite3.IntegrityError as error:
36             raise_user_integrity_error(error)
37     return user
38
39
40 def update_user(user_id, first_name=None, last_name=None, email=None, phone=None, status=None, db_path=None):
41     resolved_db_path = resolve_db_path(db_path)
42     users = load_users(db_path=resolved_db_path)
43     user = find_user_by_id(user_id, users)
44     if user is None:
45         return None
46     updated_user = build_validated_user(
47         user.user_id,
48         first_name if first_name is not None else user.first_name,
49         last_name if last_name is not None else user.last_name,
50         email if email is not None else user.email,
51         phone if phone is not None else user.phone,
52         status if status is not None else user.status,
53         user.created_at,
54         users,
55         excluded_user_id=user.user_id,
56     )
57     with closing(get_connection(resolved_db_path)) as connection:
58         try:
59             connection.execute(
60                 """
61                 UPDATE users SET first_name = ?, last_name = ?, email = ?, phone = ?, status = ?, created_at = ?
62                 WHERE id = ?
63                 """,
64                 (*user_to_record(updated_user)[1:], updated_user.user_id),
65             )
66             connection.commit()
67         except sqlite3.IntegrityError as error:
68             raise_user_integrity_error(error)
69     return updated_user
70
71
72 def delete_user(user_id, clear_parking=True, db_path=None):
73     resolved_db_path = resolve_db_path(db_path)
74     user = find_user_by_id(user_id, db_path=resolved_db_path)
75     if user is None:
76         return None
77     if clear_parking:
78         from parking_lot.parking_space_repository import clear_user_occupancy
79         clear_user_occupancy(user.user_id, db_path=resolved_db_path)

```

```

80     with closing(get_connection(resolved_db_path)) as connection:
81         connection.execute("DELETE FROM users WHERE id = ?", (user.user_id,))
82         connection.commit()
83     return user
84
85
86 def get_next_user_id(db_path=None):
87     resolved_db_path = resolve_db_path(db_path)
88     with closing(get_connection(resolved_db_path)) as connection:
89         row = connection.execute("SELECT COALESCE(MAX(id), 0) + 1 AS next_id FROM users").fetchone()
90     return row["next_id"]
91
92
93 def find_user_by_id(user_id, users=None, db_path=None):
94     normalized_user_id = int(user_id)
95     if users is not None:
96         return next((user for user in users if user.user_id == normalized_user_id), None)
97     resolved_db_path = resolve_db_path(db_path)
98     with closing(get_connection(resolved_db_path)) as connection:
99         row = connection.execute(
100             "SELECT id, first_name, last_name, email, phone, status, created_at FROM users WHERE id = ?",
101             (normalized_user_id,))
102         .fetchone()
103     return row_to_user(row) if row is not None else None
104
105
106 def build_validated_user(user_id, first_name, last_name, email, phone, status, created_at, existing_users,
107                          excluded_user_id=None):
108     normalized_email = validate_email(email)
109     ensure_unique_email(normalized_email, existing_users, excluded_user_id)
110     return User(
111         user_id,
112         validate_name(first_name, "First name"),
113         validate_name(last_name, "Last name"),
114         normalized_email,
115         validate_phone(phone),
116         validate_status(status),
117         created_at,
118     )
119
120 def ensure_unique_email(email, users, excluded_user_id=None):
121     for user in users:
122         if excluded_user_id is not None and user.user_id == int(excluded_user_id):
123             continue
124         if user.email.lower() == email.lower():
125             raise ValueError("Email already belongs to another user.")
126
127
128 def user_to_record(user):
129     return (user.user_id, user.first_name, user.last_name, user.email, user.phone, user.status, user.created_at)
130
131
132 def row_to_user(row):
133     return User(row["id"], row["first_name"], row["last_name"], row["email"], row["phone"], row["status"],
134                 row["created_at"])
135
136
137 def raise_user_integrity_error(error):
138     if "unique constraint failed: users.email" in str(error).lower():
139         raise ValueError("Email already belongs to another user.") from error
140     raise error

```

6.10 parking_lot/parking_space_repository.py

```

1 from contextlib import closing
2 from parking_lot.database import DEFAULT_DB_PATH, get_connection, resolve_db_path
3 from parking_lot.models import ParkingSpace, Vehicle, VehicleType
4 from parking_lot.validation import validate_license_plate
5
6 DEFAULT_PARKING_SPACES_DB_PATH = DEFAULT_DB_PATH
7 PARKING_SPACE_FIELDNAMES = ["building_name", "space_id", "floor", "row", "spot", "vehicle_type", "status", "user_id",
8                             "license_plate"]
9 PARKING_SPACE_SELECT_COLUMNS = """
10 ps.id AS parking_space_db_id, bs.name AS building_name, ps.space_id AS space_id,
11 bf.floor_number AS floor, pr.row_number AS "row", ps.spot_number AS spot,
12 vt.code AS vehicle_type, ps.status AS status,
13 COALESCE(CAST(ps.user_id AS TEXT), '') AS user_id,
14 COALESCE(ps.license_plate, '') AS license_plate
15 """

```

```

15 PARKING_SPACE_FROM_SQL = """
16     FROM parking_spaces ps
17     JOIN parking_rows pr ON pr.id = ps.parking_row_id
18     JOIN building_floors bf ON bf.id = pr.building_floor_id
19     JOIN building_structures bs ON bs.id = bf.building_structure_id
20     JOIN vehicle_types vt ON vt.id = ps.vehicle_type_id
21 """
22
23
24 def ensure_parking_spaces(db_path=None):
25     resolved_db_path = resolve_db_path(db_path)
26     with closing(get_connection(resolved_db_path)):
27         pass
28     return load_parking_space_rows(db_path=resolved_db_path)
29
30
31 def load_parking_space_rows(db_path=None):
32     resolved_db_path = resolve_db_path(db_path)
33     with closing(get_connection(resolved_db_path)) as connection:
34         rows = connection.execute(
35             f"SELECT {PARKING_SPACE_SELECT_COLUMNS} {PARKING_SPACE_FROM_SQL} ORDER BY bs.name, bf.floor_number,
36             pr.row_number, ps.spot_number"
37         ).fetchall()
38     return [parking_space_row_to_public_record(row) for row in rows]
39
40
41 def load_parking_spaces(building_name, db_path=None):
42     resolved_db_path = resolve_db_path(db_path)
43     spaces = []
44     for row in load_parking_space_rows(db_path=resolved_db_path):
45         if row["building_name"] != building_name:
46             continue
47         vehicle = row_to_vehicle(row, db_path=resolved_db_path) if row["status"] == "occupied" else None
48         spaces.append(row_to_parking_space(row, vehicle))
49     return spaces
50
51
52 def park_vehicle(building_name, vehicle, db_path=None):
53     resolved_db_path = resolve_db_path(db_path)
54     vehicle.license_plate = validate_license_plate(vehicle.license_plate)
55     with closing(get_connection(resolved_db_path)) as connection:
56         ensure_license_plate_available(vehicle.license_plate, connection=connection)
57         row = connection.execute(
58             f"""
59             SELECT {PARKING_SPACE_SELECT_COLUMNS} {PARKING_SPACE_FROM_SQL}
60             WHERE bs.name = ? AND vt.code = ? AND ps.status = 'available'
61             ORDER BY bf.floor_number, pr.row_number, ps.spot_number LIMIT 1
62             """
63         ).fetchone()
64         if row is None:
65             return None
66         connection.execute(
67             "UPDATE parking_spaces SET status = 'occupied', user_id = ?, license_plate = ? WHERE id = ?",
68             (vehicle.user_id if vehicle.user is not None else None, vehicle.license_plate,
69             row["parking_space_db_id"]),
70         )
71         connection.commit()
72         updated_row = fetch_parking_space_by_database_id(connection, row["parking_space_db_id"])
73     return row_to_parking_space(parking_space_row_to_public_record(updated_row), vehicle)
74
75
76 def remove_vehicle_from_space(building_name, space_id, db_path=None):
77     resolved_db_path = resolve_db_path(db_path)
78     with closing(get_connection(resolved_db_path)) as connection:
79         row = connection.execute(
80             f"SELECT {PARKING_SPACE_SELECT_COLUMNS} {PARKING_SPACE_FROM_SQL} WHERE bs.name = ? AND UPPER(ps.space_id) =
81             ? LIMIT 1",
82             (building_name, space_id.upper()),
83         ).fetchone()
84         if row is None or row["status"] != "occupied":
85             return None
86         vehicle = row_to_vehicle(parking_space_row_to_public_record(row), db_path=resolved_db_path)
87         clear_occupancy_by_database_id(connection, row["parking_space_db_id"])
88         connection.commit()
89     return vehicle
90
91
92 def remove_vehicle_by_license_plate(license_plate, db_path=None):
93     resolved_db_path = resolve_db_path(db_path)

```

```

92     normalized_license_plate = validate_license_plate(license_plate)
93     with closing(get_connection(resolved_db_path)) as connection:
94         row = connection.execute(
95             f"SELECT {PARKING_SPACE_SELECT_COLUMNS} {PARKING_SPACE_FROM_SQL} WHERE ps.status = 'occupied' AND
UPPER(ps.license_plate) = ? LIMIT 1",
96             (normalized_license_plate,),
97         ).fetchone()
98         if row is None:
99             return None
100         vehicle = row_to_vehicle(parking_space_row_to_public_record(row), db_path=resolved_db_path)
101         clear_occupancy_by_database_id(connection, row["parking_space_db_id"])
102         connection.commit()
103         return vehicle
104
105
106 def find_vehicle_by_license_plate(license_plate, db_path=None):
107     resolved_db_path = resolve_db_path(db_path)
108     normalized_license_plate = validate_license_plate(license_plate)
109     with closing(get_connection(resolved_db_path)) as connection:
110         row = connection.execute(
111             f"SELECT {PARKING_SPACE_SELECT_COLUMNS} {PARKING_SPACE_FROM_SQL} WHERE ps.status = 'occupied' AND
UPPER(ps.license_plate) = ? LIMIT 1",
112             (normalized_license_plate,),
113         ).fetchone()
114     return parking_space_row_to_public_record(row) if row is not None else None
115
116
117 def clear_user_occupancy(user_id, db_path=None):
118     resolved_db_path = resolve_db_path(db_path)
119     with closing(get_connection(resolved_db_path)) as connection:
120         result = connection.execute(
121             "UPDATE parking_spaces SET status = 'available', user_id = NULL, license_plate = NULL WHERE status =
'occupied' AND user_id = ?",
122             (int(user_id),),
123         )
124         connection.commit()
125     return result.rowcount
126
127
128 def clear_building_occupancy(building_name, db_path=None):
129     resolved_db_path = resolve_db_path(db_path)
130     with closing(get_connection(resolved_db_path)) as connection:
131         result = connection.execute(
132             f"""
133             UPDATE parking_spaces SET status = 'available', user_id = NULL, license_plate = NULL
134             WHERE status = 'occupied' AND id IN (SELECT ps.id {PARKING_SPACE_FROM_SQL} WHERE bs.name = ?)
135             """,
136             (building_name,),
137         )
138         connection.commit()
139     return result.rowcount
140
141
142 def get_building_counts(building_name, db_path=None):
143     resolved_db_path = resolve_db_path(db_path)
144     counts = {vehicle_type: {"total": 0, "occupied": 0, "available": 0} for vehicle_type in VehicleType}
145     with closing(get_connection(resolved_db_path)) as connection:
146         rows = connection.execute(
147             f"""
148             SELECT vt.code AS vehicle_type, ps.status AS status, COUNT(*) AS count
149             {PARKING_SPACE_FROM_SQL} WHERE bs.name = ? GROUP BY vt.code, ps.status
150             """,
151             (building_name,),
152         ).fetchall()
153     for row in rows:
154         vehicle_type = VehicleType.from_value(row["vehicle_type"])
155         counts[vehicle_type]["total"] += row["count"]
156         counts[vehicle_type][row["status"]] += row["count"]
157     return counts
158
159
160 def ensure_license_plate_available(license_plate, rows=None, db_path=None, connection=None):
161     if rows is not None:
162         for row in rows:
163             if row["status"] == "occupied" and row["license_plate"].upper() == license_plate:
164                 raise ValueError("License plate is already parked.")
165     return
166     owns_connection = connection is None
167     active_connection = connection or get_connection(resolve_db_path(db_path))
168     try:

```

```

169         row = active_connection.execute(
170             "SELECT 1 FROM parking_spaces WHERE status = 'occupied' AND UPPER(license_plate) = ? LIMIT 1",
171             (license_plate,),
172         ).fetchone()
173     finally:
174         if owns_connection:
175             active_connection.close()
176     if row is not None:
177         raise ValueError("License plate is already parked.")
178
179 def row_to_vehicle(row, db_path=None):
180     user = None
181     if row["user_id"]:
182         from parking_lot.user_repository import find_user_by_id
183         user = find_user_by_id(row["user_id"], db_path=resolve_db_path(db_path))
184     return Vehicle(row["vehicle_type"], user, row["license_plate"])
185
186
187 def row_to_parking_space(row, vehicle=None):
188     return ParkingSpace(row["building_name"], row["space_id"], row["floor"], row["row"], row["spot"],
189         row["vehicle_type"], row["status"], vehicle)
190
191
192 def parking_space_row_to_public_record(row):
193     return {fieldname: row[fieldname] for fieldname in PARKING_SPACE_FIELDNAMES}
194
195
196 def fetch_parking_space_by_database_id(connection, parking_space_db_id):
197     return connection.execute(
198         f"SELECT {PARKING_SPACE_SELECT_COLUMNS} {PARKING_SPACE_FROM_SQL} WHERE ps.id = ? LIMIT 1",
199         (parking_space_db_id,),
200     ).fetchone()
201
202
203 def clear_occupancy_by_database_id(connection, parking_space_db_id):
204     connection.execute("UPDATE parking_spaces SET status = 'available', user_id = NULL, license_plate = NULL WHERE id =
        ?", (parking_space_db_id,))

```

6.11 parking_lot/building.py

```

1 from parking_lot.models import Vehicle
2 from parking_lot.parking_space_repository import (
3     DEFAULT_PARKING_SPACES_DB_PATH,
4     clear_building_occupancy,
5     get_building_counts,
6     load_parking_spaces,
7     park_vehicle,
8     remove_vehicle_from_space,
9 )
10 from parking_lot.structures import get_structure_names
11
12
13 class ParkingBuilding:
14     def __init__(self, building_name, db_path=DEFAULT_PARKING_SPACES_DB_PATH):
15         if building_name not in get_structure_names():
16             raise ValueError(f"Unknown parking building structure: {building_name}")
17         self.building_name = building_name
18         self.db_path = db_path
19         self.parking_spaces = []
20         self.reload()
21
22     def reload(self):
23         self.parking_spaces = load_parking_spaces(self.building_name, db_path=self.db_path)
24         return self.parking_spaces
25
26     def get_parking_spaces(self):
27         return self.parking_spaces
28
29     def iter_spaces(self):
30         yield from self.parking_spaces
31
32     def find_space_by_id(self, space_id):
33         normalized_space_id = space_id.strip().upper()
34         return next((space for space in self.iter_spaces() if space.space_id.upper() == normalized_space_id), None)
35
36     def find_available_space(self, vehicle: Vehicle):
37         return next((space for space in self.iter_spaces() if space.can_park(vehicle)), None)
38
39     def park_vehicle(self, vehicle: Vehicle):

```

```

40     space = park_vehicle(self.building_name, vehicle, db_path=self.db_path)
41     self.reload()
42     return space
43
44     def remove_vehicle_from_space(self, space_id):
45         vehicle = remove_vehicle_from_space(self.building_name, space_id, db_path=self.db_path)
46         self.reload()
47         return vehicle
48
49     def clear_occupancy(self):
50         cleared_count = clear_building_occupancy(self.building_name, db_path=self.db_path)
51         self.reload()
52         return cleared_count
53
54     def get_space_counts(self):
55         return get_building_counts(self.building_name, db_path=self.db_path)

```

6.12 parking_lot/demo_seed.py

```

1  from __future__ import annotations
2
3  from contextlib import closing
4  from dataclasses import dataclass
5  from parking_lot.database import get_connection, resolve_db_path
6  from parking_lot.models import VehicleType
7
8  DEMO_USERS = [
9      (1001, "Aarav", "Sharma", "aarav.demo@example.com", "+91-98765-43210", "active", "2026-01-15"),
10     (1002, "Priya", "Nair", "priya.demo@example.com", "+91-98765-43211", "active", "2026-02-03"),
11     (1003, "Rohan", "Mehta", "rohan.demo@example.com", "+91-98765-43212", "inactive", "2026-02-18"),
12     (1004, "Ananya", "Rao", "ananya.demo@example.com", "+91-98765-43213", "active", "2026-03-01"),
13     (1005, "Vikram", "Singh", "vikram.demo@example.com", "+91-98765-43214", "active", "2026-03-12"),
14     (1006, "Neha", "Patel", "neha.demo@example.com", "+91-98765-43215", "pending", "2026-04-05"),
15     (1007, "Kabir", "Khan", "kabir.demo@example.com", "+91-98765-43216", "active", "2026-04-21"),
16     (1008, "Isha", "Gupta", "isha.demo@example.com", "+91-98765-43217", "inactive", "2026-05-08"),
17     (1009, "Arjun", "Iyer", "arjun.demo@example.com", "+91-98765-43218", "active", "2026-05-19"),
18     (1010, "Meera", "Joshi", "meera.demo@example.com", "+91-98765-43219", "pending", "2026-06-02"),
19 ]
20 DEMO_USER_IDS = tuple(user[0] for user in DEMO_USERS)
21 DEMO_EMAILS = tuple(user[3] for user in DEMO_USERS)
22 DEMO_PLATES = tuple(f"DEMO-{{plate_number:03d}}" for plate_number in range(1, 31))
23 DEMO_SPACE_TARGETS = [
24     ("small", VehicleType.BIKE, 2), ("small", VehicleType.CAR_MINI, 2), ("small", VehicleType.CAR_SUV, 1),
25     ("mixed", VehicleType.BIKE, 2), ("mixed", VehicleType.CAR_MINI, 3), ("mixed", VehicleType.CAR_SUV, 2),
26     ("mixed", VehicleType.TRUCK_MINI, 2), ("mixed", VehicleType.TRUCK_LARGE, 1),
27     ("truck_friendly", VehicleType.CAR_MINI, 1), ("truck_friendly", VehicleType.CAR_SUV, 1),
28     ("truck_friendly", VehicleType.TRUCK_MINI, 3), ("truck_friendly", VehicleType.TRUCK_LARGE, 3),
29     ("very_large_complex", VehicleType.BIKE, 2), ("very_large_complex", VehicleType.CAR_MINI, 2),
30     ("very_large_complex", VehicleType.CAR_SUV, 1), ("very_large_complex", VehicleType.TRUCK_MINI, 1),
31     ("very_large_complex", VehicleType.TRUCK_LARGE, 1),
32 ]
33
34
35 @dataclass(frozen=True)
36 class DemoSeedResult:
37     users_seeded: int
38     occupied_spaces_seeded: int
39
40
41 def seed_demo_data(db_path=None):
42     resolved_db_path = resolve_db_path(db_path)
43     with closing(get_connection(resolved_db_path)) as connection:
44         clear_demo_occupancy(connection)
45         reset_demo_users(connection)
46         seeded_spaces = seed_demo_occupancy(connection)
47         connection.commit()
48     return DemoSeedResult(len(DEMO_USERS), seeded_spaces)
49
50
51 def clear_demo_occupancy(connection):
52     connection.execute(
53         f"""
54         UPDATE parking_spaces SET status = 'available', user_id = NULL, license_plate = NULL
55         WHERE user_id IN ({placeholders(DEMO_USER_IDS)}) OR license_plate IN ({placeholders(DEMO_PLATES)})
56         """
57     )
58
59
60
61 def reset_demo_users(connection):

```

```

62     connection.execute(
63         f"DELETE FROM users WHERE id IN ({placeholders(DEMO_USER_IDS)}) OR email IN ({placeholders(DEMO_EMAILS)})",
64         DEMO_USER_IDS + DEMO_EMAILS,
65     )
66     connection.executemany(
67         "INSERT INTO users (id, first_name, last_name, email, phone, status, created_at) VALUES (?, ?, ?, ?, ?, ?, ?)",
68         DEMO_USERS,
69     )
70
71
72 def seed_demo_occupancy(connection):
73     seeded_count = 0
74     for building_name, vehicle_type, count in DEMO_SPACE_TARGETS:
75         for parking_space_id in get_available_space_ids(connection, building_name, vehicle_type, count):
76             user_id = DEMO_USER_IDS[seeded_count % len(DEMO_USER_IDS)]
77             license_plate = DEMO_PLATES[seeded_count]
78             connection.execute(
79                 "UPDATE parking_spaces SET status = 'occupied', user_id = ?, license_plate = ? WHERE id = ?",
80                 (user_id, license_plate, parking_space_id),
81             )
82             seeded_count += 1
83     return seeded_count
84
85
86 def get_available_space_ids(connection, building_name, vehicle_type, count):
87     rows = connection.execute(
88         """
89         SELECT ps.id FROM parking_spaces ps
90         JOIN parking_rows pr ON pr.id = ps.parking_row_id
91         JOIN building_floors bf ON bf.id = pr.building_floor_id
92         JOIN building_structures bs ON bs.id = bf.building_structure_id
93         JOIN vehicle_types vt ON vt.id = ps.vehicle_type_id
94         WHERE bs.name = ? AND vt.code = ? AND ps.status = 'available'
95         ORDER BY bf.floor_number, pr.row_number, ps.spot_number LIMIT ?
96         """,
97         (building_name, vehicle_type.name, count),
98     ).fetchall()
99     if len(rows) < count:
100         raise ValueError(f"Not enough available {vehicle_type.name} spaces in {building_name} to seed demo data.")
101     return [row["id"] for row in rows]
102
103
104 def placeholders(values):
105     return ", ".join("?" for _ in values)

```

6.13 parking_lot/cli.py

```

1 from rich import box
2 from rich.align import Align
3 from rich.console import Console
4 from rich.panel import Panel
5 from rich.prompt import Confirm, Prompt
6 from rich.table import Table
7 from parking_lot.building import ParkingBuilding
8 from parking_lot.demo_seed import seed_demo_data
9 from parking_lot.models import Vehicle, VehicleType
10 from parking_lot.parking_space_repository import clear_building_occupancy, find_vehicle_by_license_plate,
11     remove_vehicle_by_license_plate
12 from parking_lot.structures import PARKING_BUILDING_STRUCTURES, count_vehicle_types, get_structure_names
13 from parking_lot.user_repository import create_user, delete_user, find_user_by_id, load_users, update_user
14 from parking_lot.validation import VALID_USER_STATUSES, validate_license_plate
15
16 VEHICLE_TYPE_LABELS = {
17     VehicleType.BIKE: "Bike", VehicleType.CAR_MINI: "Mini Car", VehicleType.CAR_SUV: "SUV Car",
18     VehicleType.TRUCK_MINI: "Mini Truck", VehicleType.TRUCK_LARGE: "Large Truck",
19 }
20 MENU_OPTIONS = {
21     "1": "Dashboard", "2": "List users", "3": "Create user", "4": "Edit user", "5": "Delete user",
22     "6": "Park vehicle", "7": "Occupied spaces", "8": "Find vehicle by plate", "9": "Remove vehicle",
23     "10": "Show building structures", "11": "Switch building", "12": "Clear selected building",
24     "13": "Seed demo data", "14": "Exit",
25 }
26
27 def run_cli():
28     ParkingLotCLI().run()
29
30
31 class ParkingLotCLI:
32     def __init__(self, console=None):

```

```

33     self.console = console or Console()
34     self.structure_name = None
35     self.building = None
36
37 def run(self):
38     self.show_welcome()
39     self.select_starting_building()
40     while True:
41         self.show_header()
42         self.show_menu()
43         choice = Prompt.ask("Choose an option", choices=list(MENU_OPTIONS.keys()), console=self.console)
44         if choice == "1": self.show_dashboard()
45         elif choice == "2": self.show_users()
46         elif choice == "3": self.create_user_from_input()
47         elif choice == "4": self.edit_user_from_input()
48         elif choice == "5": self.delete_user_from_input()
49         elif choice == "6": self.park_vehicle_from_input()
50         elif choice == "7": self.show_occupied_spaces()
51         elif choice == "8": self.find_vehicle_from_input()
52         elif choice == "9": self.remove_vehicle_from_input()
53         elif choice == "10": self.show_building_structures()
54         elif choice == "11": self.switch_building_structure()
55         elif choice == "12": self.clear_selected_building()
56         elif choice == "13": self.seed_demo_data_from_input()
57         elif choice == "14":
58             self.console.print("[bold green]Goodbye.[/bold green]")
59             break
60
61 def show_welcome(self):
62     self.console.print(Panel(Align.center("[bold cyan]Parking Lot Manager[/bold cyan]\n[white]Choose a building,
manage users, and persist parked vehicles.[/white]"), border_style="cyan", box=box.ROUNDED))
63
64 def select_starting_building(self):
65     self.console.print("[bold>Select the building to operate on first.[/bold]")
66     self.structure_name = self.choose_building_structure(allow_cancel=False)
67     self.building = ParkingBuilding(self.structure_name)
68
69 def show_header(self):
70     counts = self.building.get_space_counts()
71     total = sum(c["total"] for c in counts.values())
72     occupied = sum(c["occupied"] for c in counts.values())
73     available = sum(c["available"] for c in counts.values())
74     header = f"[bold]Building: [/bold] {self.structure_name}    [green]Available: [/green] {available}
[yellow]Occupied: [/yellow] {occupied}    [cyan]Total: [/cyan] {total}"
75     self.console.print(Panel(header, title="Parking Lot CLI", border_style="cyan"))
76
77 def show_menu(self):
78     table = Table(box=box.SIMPLE, show_header=False, padding=(0, 2))
79     table.add_column("Option", style="cyan", justify="right")
80     table.add_column("Action", style="white")
81     for key, label in MENU_OPTIONS.items():
82         table.add_row(key, label)
83     self.console.print(table)
84
85 def show_dashboard(self):
86     self.show_parking_summary()
87     self.show_occupied_spaces(limit=10)
88
89 def show_users(self):
90     users = load_users()
91     if not users:
92         self.console.print("[yellow]No users found.[/yellow]")
93         return
94     self.console.print(self.build_users_table(users))
95
96 def create_user_from_input(self):
97     self.console.print(Panel("Create User", border_style="cyan"))
98     while True:
99         try:
100             user = create_user(self.prompt_required("First name"), self.prompt_required("Last name"),
self.prompt_required("Email"), self.prompt_required("Phone"), status=self.prompt_status(default="active"))
101             except ValueError as error:
102                 self.console.print(f"[red]{error}[/red]")
103                 if not Confirm.ask("Try again?", default=True, console=self.console): return
104             else:
105                 self.console.print(f"[green]Created user {user.user_id}: {user.get_full_name()}[/green]")
106                 return
107
108 def edit_user_from_input(self):
109     user = self.choose_user(load_users())

```

```

110     if user is None: return
111     self.console.print("[dim]Press Enter to keep the current value.[/dim]")
112     try:
113         updated_user = update_user(
114             user.user_id,
115             first_name=self.prompt_optional("First name", user.first_name),
116             last_name=self.prompt_optional("Last name", user.last_name),
117             email=self.prompt_optional("Email", user.email),
118             phone=self.prompt_optional("Phone", user.phone),
119             status=self.prompt_status(default=user.status),
120         )
121     except ValueError as error:
122         self.console.print(f"[red]{error}[/red]")
123         return
124     self.console.print(f"[green]Updated user {updated_user.user_id}: {updated_user.get_full_name()}[/green]")
125
126 def delete_user_from_input(self):
127     user = self.choose_user(load_users())
128     if user is None: return
129     if not Confirm.ask(f"Delete {user.get_full_name()} and clear their parked vehicles?", default=False,
130 console=self.console):
131         self.console.print("[yellow>Delete cancelled.[/yellow]")
132         return
133     deleted_user = delete_user(user.user_id)
134     self.console.print(f"[green]Deleted {deleted_user.get_full_name()} and cleared their spaces.[/green]")
135
136 def show_building_structures(self):
137     table = Table(title="Parking Building Structures", box=box.ROUNDED)
138     table.add_column("Option", style="cyan", justify="right")
139     table.add_column("Name", style="bold")
140     table.add_column("Spaces", justify="right")
141     table.add_column("Breakdown")
142     for index, name in enumerate(get_structure_names(), start=1):
143         counts = count_vehicle_types(PARKING_BUILDING_STRUCTURES[name])
144         table.add_row(str(index), name, str(sum(counts.values())) , self.format_vehicle_type_counts(counts))
145     self.console.print(table)
146
147 def switch_building_structure(self):
148     structure_name = self.choose_building_structure(allow_cancel=True)
149     if structure_name is None: return
150     self.structure_name = structure_name
151     self.building = ParkingBuilding(self.structure_name)
152     self.console.print(f"[green]Switched to '{self.structure_name}'.[/green]")
153
154 def show_parking_summary(self):
155     counts = self.building.get_space_counts()
156     table = Table(title="Parking Summary", box=box.ROUNDED)
157     table.add_column("Vehicle Type", style="cyan")
158     table.add_column("Available", justify="right", style="green")
159     table.add_column("Occupied", justify="right", style="yellow")
160     table.add_column("Total", justify="right")
161     for vehicle_type in VehicleType:
162         c = counts[vehicle_type]
163         table.add_row(VEHICLE_TYPE_LABELS[vehicle_type], str(c["available"]), str(c["occupied"]), str(c["total"]))
164     self.console.print(table)
165
166 def park_vehicle_from_input(self):
167     users = load_users()
168     if not users:
169         self.console.print("[yellow]No users found. Create a user before parking.[/yellow]")
170         return
171     user = self.choose_user(users)
172     if user is None: return
173     license_plate = self.prompt_license_plate()
174     if license_plate is None: return
175     vehicle_type = self.choose_vehicle_type()
176     if vehicle_type is None: return
177     try:
178         space = self.building.park_vehicle(Vehicle(vehicle_type, user, license_plate))
179     except ValueError as error:
180         self.console.print(f"[red]{error}[/red]")
181         return
182     if space is None:
183         self.console.print(f"[yellow]No available {VEHICLE_TYPE_LABELS[vehicle_type]} spaces in {self.structure_name}.[/yellow]")
184         return
185     self.console.print(Panel(f"[green]Allocated {space.space_id}[/green]\nVehicle: {VEHICLE_TYPE_LABELS[vehicle_type]}\nPlate: {license_plate}\nUser: {user.get_full_name()}", title="Vehicle Parked", border_style="green"))

```

```

186
187 def show_occupied_spaces(self, limit=None):
188     occupied_spaces = [space for space in self.building.iter_spaces() if not space.is_available()]
189     if not occupied_spaces:
190         self.console.print("[yellow]No occupied spaces.[/yellow]")
191         return
192     table = Table(title="Occupied Spaces", box=box.ROUNDED)
193     table.add_column("Space", style="cyan")
194     table.add_column("Vehicle")
195     table.add_column("Plate")
196     table.add_column("User")
197     table.add_column("User ID", justify="right")
198     for space in occupied_spaces[:limit] if limit is not None else occupied_spaces:
199         vehicle = space.parked_vehicle
200         user = vehicle.user if vehicle is not None else None
201         table.add_row(space.space_id, VEHICLE_TYPE_LABELS[space.vehicle_type], vehicle.license_plate if vehicle else
202         "", user.get_full_name() if user else "Unknown user", str(user.user_id) if user else "")
203     self.console.print(table)
204     if limit is not None and len(occupied_spaces) > limit:
205         self.console.print(f"[dim]Showing {limit} of {len(occupied_spaces)} occupied spaces.[/dim]")
206
207 def find_vehicle_from_input(self):
208     license_plate = self.prompt_license_plate()
209     if license_plate is None: return
210     row = find_vehicle_by_license_plate(license_plate)
211     if row is None:
212         self.console.print("[yellow]No parked vehicle found for that plate.[/yellow]")
213         return
214     user = find_user_by_id(row["user_id"]) if row["user_id"] else None
215     self.console.print(Panel(f"Building: {row['building_name']}\nSpace: {row['space_id']}\nVehicle:
216     {VEHICLE_TYPE_LABELS[VehicleType.from_value(row['vehicle_type'])]\nPlate: {row['license_plate']}\nUser:
217     {user.get_full_name() if user else 'Unknown user'}", title="Vehicle Found", border_style="green"))
218
219 def remove_vehicle_from_input(self):
220     choice = Prompt.ask("Remove by", choices=["space", "plate", "cancel"], default="space", console=self.console)
221     if choice == "cancel":
222         self.console.print("[yellow]Remove cancelled.[/yellow]")
223         return
224     if choice == "plate":
225         self.remove_vehicle_by_plate_from_input()
226         return
227     space = self.choose_occupied_space()
228     if space is None: return
229     self.print_removed_vehicle(self.building.remove_vehicle_from_space(space.space_id), space.space_id)
230
231 def remove_vehicle_by_plate_from_input(self):
232     license_plate = self.prompt_license_plate()
233     if license_plate is None: return
234     row = find_vehicle_by_license_plate(license_plate)
235     if row is None:
236         self.console.print("[yellow]No parked vehicle found for that plate.[/yellow]")
237         return
238     vehicle = remove_vehicle_by_license_plate(license_plate)
239     self.building.reload()
240     self.print_removed_vehicle(vehicle, row["space_id"])
241
242 def clear_selected_building(self):
243     if not Confirm.ask(f"Clear all occupied spaces in {self.structure_name}?", default=False, console=self.console):
244         self.console.print("[yellow]Clear cancelled.[/yellow]")
245         return
246     cleared_count = clear_building_occupancy(self.structure_name)
247     self.building.reload()
248     self.console.print(f"[green]Cleared {cleared_count} occupied spaces.[/green]")
249
250 def seed_demo_data_from_input(self):
251     if not Confirm.ask("Reset demo users and demo parking occupancy?", default=False, console=self.console):
252         self.console.print("[yellow]Demo seed cancelled.[/yellow]")
253         return
254     result = seed_demo_data()
255     self.building.reload()
256     self.console.print(f"[green]Seeded {result.users_seeded} demo users and {result.occupied_spaces_seeded} occupied
257     spaces.[/green]")
258
259 def choose_building_structure(self, allow_cancel):
260     names = get_structure_names()
261     self.show_building_structures()
262     while True:
263         prompt = "Structure name or number" + (" (blank to cancel)" if allow_cancel else "")
264         choice = Prompt.ask(prompt, default="", show_default=False, console=self.console).strip()
265         if not choice and allow_cancel:

```

```

262         self.console.print("[yellow]Structure selection cancelled.[/yellow]")
263         return None
264     if choice.isdigit() and 0 <= int(choice) - 1 < len(names):
265         return names[int(choice) - 1]
266     if choice in PARKING_BUILDING_STRUCTURES:
267         return choice
268     self.console.print("[red]Unknown building structure. Try again.[/red]")
269
270 def choose_user(self, users):
271     if not users:
272         self.console.print("[yellow]No users found.[/yellow]")
273         return None
274     self.console.print(self.build_users_table(users))
275     while True:
276         user_id_input = Prompt.ask("User id (blank to cancel)", default="", show_default=False,
console=self.console).strip()
277         if not user_id_input:
278             self.console.print("[yellow]User selection cancelled.[/yellow]")
279             return None
280         if not user_id_input.isdigit():
281             self.console.print("[red]User id must be a number.[/red]")
282             continue
283         user = find_user_by_id(user_id_input, users)
284         if user is not None: return user
285         self.console.print("[red]User not found. Try again.[/red]")
286
287 def choose_vehicle_type(self):
288     vehicle_types = list(VehicleType)
289     table = Table(title="Vehicle Types", box=box.ROUNDED)
290     table.add_column("Option", justify="right", style="cyan")
291     table.add_column("Vehicle Type")
292     for index, vehicle_type in enumerate(vehicle_types, start=1):
293         table.add_row(str(index), VEHICLE_TYPE_LABELS[vehicle_type])
294     self.console.print(table)
295     while True:
296         choice = Prompt.ask("Vehicle type number (blank to cancel)", default="", show_default=False,
console=self.console).strip()
297         if not choice: return None
298         if choice.isdigit() and 0 <= int(choice) - 1 < len(vehicle_types): return vehicle_types[int(choice) - 1]
299         self.console.print("[red]Unknown vehicle type. Try again.[/red]")
300
301 def choose_occupied_space(self):
302     occupied_spaces = [space for space in self.building.iter_spaces() if not space.is_available()]
303     if not occupied_spaces:
304         self.console.print("[yellow]No occupied spaces.[/yellow]")
305         return None
306     self.show_occupied_spaces()
307     while True:
308         space_id = Prompt.ask("Space id (blank to cancel)", default="", show_default=False,
console=self.console).strip()
309         if not space_id: return None
310         space = self.building.find_space_by_id(space_id)
311         if space is None: self.console.print("[red]Unknown parking space. Try again.[/red]")
312         elif space.is_available(): self.console.print(f"[yellow]{space.space_id} is already available.[/yellow]")
313         else: return space
314
315 def build_users_table(self, users):
316     table = Table(title="Users", box=box.ROUNDED)
317     table.add_column("ID", justify="right", style="cyan")
318     table.add_column("Name")
319     table.add_column("Email")
320     table.add_column("Phone")
321     table.add_column("Status")
322     for user in users:
323         table.add_row(str(user.user_id), user.get_full_name(), user.email, user.phone, user.status)
324     return table
325
326 def prompt_required(self, prompt):
327     while True:
328         value = Prompt.ask(prompt, console=self.console).strip()
329         if value: return value
330         self.console.print("[red]This field is required.[/red]")
331
332 def prompt_optional(self, prompt, current_value):
333     value = Prompt.ask(f"{prompt} [{current_value}]", default="", show_default=False, console=self.console).strip()
334     return value or current_value
335
336 def prompt_status(self, default):
337     return Prompt.ask("Status", choices=sorted(VALID_USER_STATUSES), default=default, console=self.console)
338

```

```
339 def prompt_license_plate(self):
340     while True:
341         value = Prompt.ask("License plate (blank to cancel)", default="", show_default=False,
342                             console=self.console).strip()
343         if not value: return None
344         try: return validate_license_plate(value)
345         except ValueError as error: self.console.print(f"[red]{error}[/red]")
346
347 def print_removed_vehicle(self, vehicle, space_id):
348     if vehicle is None:
349         self.console.print("[yellow]No parked vehicle found to remove.[/yellow]")
350         return
351     user = vehicle.user
352     suffix = f" for {user.get_full_name()}" if user is not None else ""
353     self.console.print(f"[green]Removed {VEHICLE_TYPE_LABELS[vehicle.vehicle_type]} {vehicle.license_plate} from {space_id}{suffix}[/green]")
354
355 def format_vehicle_type_counts(self, counts):
356     return ", ".join(f"{counts.get(v, 0)} {VEHICLE_TYPE_LABELS[v]}" for v in VehicleType if counts.get(v, 0))
```